

EXAMPLE_PROJECTS_QUICK_REFERENCE

Quick Reference: Example Projects Summary

Project Overview

Project	Qwen Code	Gemini CLI	DeepSeek CLI
Repository	github.com/ QwenLM/qwen-code	github.com/ google-gemini/ gemini-cli	github.com/ holasoyalva/ deepseek-cli
Language	TypeScript	TypeScript	TypeScript
Node Version	20+	20+	18+
License	Apache 2.0	Apache 2.0	MIT
Complexity	High	High	Low
Dependencies	Many	Many	5 core

Core Features Summary

Qwen Code - Best For: Multimodal & Chinese Regions

Key Strengths: - Vision model auto-switching (unique feature) - DashScope token caching - Qwen OAuth with 2,000 req/day free - Session token limits - Parser optimized for Qwen models

LLM Providers: - Qwen OAuth (primary) - OpenAI-compatible APIs - Regional: Alibaba Cloud, ModelScope, OpenRouter

Supported Models: - qwen3-coder-plus (latest) - qwen3-vl-plus (vision) - Variants: 30BA3B, 480B-A35B-Instruct

Unique Features to Port: 1. Vision model auto-detection and switching 2. Three modes: once, session, persist 3. DashScope cache control headers 4. Ephemeral token caching 5. Session and prompt ID tracking

Gemini CLI - Best For: Enterprise & Search Grounding

Key Strengths: - Google Search grounding (real-time info) - Full checkpointing system - Three auth options (OAuth, API Key, Vertex AI) - Thinking mode with token limits - MCP protocol support - Loop detection - Chat compression

LLM Providers: - Google OAuth (60 req/min, 1k req/day free) - Gemini API Key (100 req/day free) - Vertex AI (enterprise)

Supported Models: - gemini-2.5-pro (default) - gemini-2.5-flash (fast) - gemini-2.5-flash-lite (lightweight) - auto (automatic selection)

Unique Features to Port: 1. Google Search integration 2. Checkpointing and session save/restore 3. Thinking mode (8,192 token limit) 4. Model fallback strategy for degradation 5. Loop detection service 6. Chat compression for context optimization 7. IDE context tracking 8. Turn-based conversation management

DeepSeek CLI - Best For: Local-First & Simplicity

Key Strengths: - Dual local/cloud architecture - Ollama integration - Minimal dependencies (5 core) - Simple, clean codebase - Setup automation - Health checking

LLM Providers: - Local Ollama (default, free, private) - DeepSeek Cloud API

Supported Models: - deepseek-coder:1.3b (lightweight, 2GB RAM) - deepseek-coder:6.7b (recommended, 8GB RAM) - deepseek-coder:33b (most capable, 32GB RAM)

Unique Features to Port: 1. Dual local/cloud execution modes 2. Ollama health checks 3. Model availability detection 4. Setup automation command 5. Minimal dependency footprint 6. Response formatting with syntax highlighting 7. Graceful fallback logic

Porting Priority Matrix

High Priority (Implement First)

1. **Vision Model Auto-Switching** (Qwen)
 - Detects images in prompts
 - Switches to vision model automatically
 - User configurable (once/session/persist)
2. **Google Search Grounding** (Gemini)
 - Real-time information integration

- Production-ready implementation
- 3. **MCP Protocol Support** (Qwen/Gemini)
 - Custom tool integration
 - Extensible architecture
- 4. **Token Caching Strategies** (Qwen/Gemini)
 - Ephemeral cache control
 - Cost optimization
- 5. **Checkpointing System** (Gemini)
 - Save/restore conversation state
 - Work preservation

Medium Priority (Implement Next)

1. **Dual Local/Cloud Architecture** (DeepSeek)
2. **Model Fallback Strategies** (Gemini)
3. **Loop Detection Service** (Gemini)
4. **Response Formatting** (DeepSeek)
5. **Session Token Limits** (Qwen)

Lower Priority (Nice to Have)

1. **IDE Companion** (Gemini)
2. **Telemetry** (Gemini/Qwen)
3. **Release Automation** (Gemini)
4. **GitHub Actions** (Gemini)

Implementation Patterns

Authentication Pattern (All Three)

Qwen: OAuth (primary) → API Key (fallback)
Gemini: OAuth → API Key → Vertex AI
DeepSeek: Local (default) → Cloud API (fallback)

Provider Configuration

Environment Variables:

- API keys/tokens
- Base URLs
- Model selection
- Feature flags

Config Files:

- ~/.qwen/settings.json
- ~/.gemini/settings.json
- .env in project

CLI Flags:

- Override env vars
- One-time settings

Error Handling Pattern

- Connection errors: Detailed diagnostics
 - Auth errors: Clear guidance
 - Model errors: Installation commands
 - Setup errors: Automated recovery
-

Key Technical Details

Qwen Code

- **File:** packages/core/src/core/openaiContentGenerator/provider/dashscope.ts
- **Cache Strategy:** System message + last tool + latest history
- **Vision Detection:** Matches 'vision-model', 'qwen-vl', 'qwen3-vl-plus'
- **Token Limit:** Enforced per model, respects DashScope limits

Gemini CLI

- **Client:** packages/core/src/core/client.ts
- **Thinking:** 8,192 token limit, disabled for lite models
- **Fallback:** Pro → Flash (preserves lite models)
- **Turns:** Max 100 per session
- **Compression:** Auto-triggered for long contexts

DeepSeek CLI

- **API:** src/api.ts with dual-mode execute
 - **Modes:** Local (timeout: 120s) vs Cloud (timeout: 30s)
 - **Setup:** src/commands/setup.ts with verification
 - **Format:** src/commands/interactive.ts with markdown rendering
-

File Locations

Qwen Code

- Models: `packages/cli/src/ui/models/availableModels.ts`
- DashScope: `packages/core/src/core/openaiContentGenerator/provider/dashscope.ts`
- Auth: `packages/core/src/` (OAuth & credential storage)

Gemini CLI

- Models: `packages/core/src/config/models.ts`
- Client: `packages/core/src/core/client.ts`
- Requests: `packages/core/src/core/geminiRequest.ts`
- Auth: `packages/core/src/` (OAuth, API key, Vertex AI)

DeepSeek CLI

- Config: `src/config.ts`
- API: `src/api.ts`
- CLI: `src/cli.ts`
- Commands: `src/commands/` (chat, interactive, setup)

Recommended Import Order

1. **Start with DeepSeek** for baseline provider pattern
2. **Add Qwen** for vision and caching complexity
3. **Integrate Gemini** for search and advanced features
4. **Implement MCP** for extensibility
5. **Add specialized services** (loop detection, compression)

Configuration Structure

Environment-Based

```
# Qwen
OPENAI_API_KEY=...
OPENAI_BASE_URL=...
OPENAI_MODEL=...
```

```
# Gemini
GEMINI_API_KEY=...
```

```
GOOGLE_CLOUD_PROJECT=...
GOOGLE_GENAI_USE_VERTEXAI=...
```

```
# DeepSeek
DEEPSEEK_USE_LOCAL=true
DEEPSEEK_MODEL=...
DEEPSEEK_API_KEY=...
OLLAMA_HOST=...
```

Settings File-Based

```
{
  "sessionTokenLimit": 32000,
  "experimental": {
    "vlmSwitchMode": "once",
    "visionModelPreview": true,
    "disableCacheControl": false
  }
}
```

CLI Flag-Based

```
--api-key <key>
--model <model>
--local / --cloud
--base-url <url>
--vlm-switch-mode {once|session|persist}
```

Testing Strategy

Qwen Code

- Test vision detection with sample images
- Test model switching modes
- Test cache control headers
- Test token limit enforcement

Gemini CLI

- Test thinking mode with/without support
- Test fallback logic during degradation
- Test loop detection
- Test compression triggers

- Test checkpointing

DeepSeek CLI

- Test Ollama connection checking
 - Test local model detection
 - Test cloud fallback
 - Test response formatting
 - Test setup automation
-

Deployment Considerations

Qwen Code: Regional endpoints, OAuth tokens, DashScope headers **Gemini CLI:** OAuth flow, Vertex AI auth, service account impersonation **DeepSeek CLI:** Ollama service requirement, model downloading, health monitoring

Documentation Links

- Qwen Code Docs: `cli_agents/qwen-code/docs/`
- Gemini CLI Docs: `cli_agents/gemini-cli/docs/`
- DeepSeek CLI Docs: `cli_agents/deepseek-cli/`

Full analysis available in: `/Users/milosvasic/Projects/helix_code/EXAMPLE_PROJECTS_ANALYSIS.md`